# Project Management Module

## Software Requirement Specification

Prepared by : Nardos Regassa
Version: 1.0
Date: October 27, 2026

**Table of content…….Page**


1. Introduction……. 1
2. General Description…… 1
3. Functional Requirements…… 3
4. Use case…… 8
5. Non-Functional Attributes…….. 9
6. External Interface Requirement……. 10
7. Data Requirment…… 11
8. Release Plan…….. 12
9. Risk and Mitigation…….. 12
10. Acceptance and Verification……. 12
11. Open questions for stakeholders…….. 13
12. Appendices…….. 13
13. Conclusion…… 14

# Introduction

## 1.1 Purpose

This document specifies the requirements for Project Management module, a web-based system that helps project managers and product managers plan software projects, break them down into tasks, assign those tasks to specific developers, track development progress through GitHub, and keep developers on schedule through automated Telegram and SMS reminders.

## 1.2 Business Context and Problem Statement

Delivery managers currently rely on a mix of chat messages, spreadsheets and manual checkins to know who is working on what. This leads to unclear ownership, missed deadlines, late discovery of blockers, and no objective link between planned tasks and actual code changes. Reviewers and controllers have no consolidated view of developer output, and developers are reminded of deadlines inconsistently.

This system closes these gaps by connecting planning (projects, teams, tasks, deadlines) with execution (commits, pull requests, review status) and communication (Telegram, SMS and other reminders) in one auditable system.

## 1.3 Product Vision

A reliable, secure and scalable platform that any engineering organization can adopt to plan work, assign accountability to specific people, see delivery progress in real time, and act on risk before deadlines are missed.

## 1.4 Scope

**In scope:**

- Project intake with detailed descriptions, goals, milestones and deadlines.
- Task breakdown and assignment to specific people within Frontend, Backend and UI/UX teams, with individual and team deadlines.
- Git integration (GitHub first) for commits, branches, pull requests and review status.
- Reviewer and controller views of developer code activity.
- Configurable progress-update cycles (for example every 3 or 7 days).
- Notification engine for Telegram, SMS, email and in-app, with reminders and escalation rules.
- Dashboards, risk indicators, reports and exports.
- Audit logging, administration, and a public REST API with webhooks.

**Out of scope (initial release):** payroll, time-and-billing, built-in code editor or IDE, CI/CD pipeline execution, resource-cost budgeting, and Git providers other than GitHub (designed to be added later).

# General Description

## 2.1 Product Perspective

This system is a web-based, internal engineering delivery platform. It integrates with external services: GitHub for repository activity, the Telegram Bot API and an SMS gateway

for notifications, an email service, and optionally an identity provider (SSO) and an AI/LLM service for task suggestions. It also exposes its own REST API and outgoing webhooks for integration with other company tools.

## 2.2 User Class

| User class | Description | Main needs |
|---|---|---|
| Admin | Single administrator role for the company: manages users, roles, integrations, skill catalog, policies and audit. | Users, roles, integrations, policies, audit. |
| Project / Product Manager (Controller) | Owns projects and delivery. | Plan, assign, monitor progress, escalate, report. |
| Team Lead | Leads a Frontend, Backend,AI or UI/UX team. | Distribute work, monitor team, review updates. |
| Reviewer | Reviews code and deliverables. | See commits and pull requests, approve or reject, comment. |
| Developer | Frontend, Backend , AI or UI/UX engineer. | See tasks and deadlines, submit updates, get reminders. |
| Executive / Viewer | Read-only stakeholder. | Portfolio dashboards and reports. |

## 2.3 Operating Environment

Modern desktop and mobile browsers (current and previous major versions of Chrome, Edge, Firefox, Safari). Server components run in a containerized cloud or on-premises environment.

## 2.4 Assumptions and Dependencies

- Engineers have GitHub accounts, Telegram accounts and valid mobile numbers, and consent to notifications.
- The company grants this system access to its repositories through an approved GitHub App or OAuth integration.
- A commercial or local SMS provider with acceptable delivery coverage and cost is available.
- Third-party APIs (GitHub, Telegram, SMS, email) remain available and within rate limits.
- Management will enforce use of the system, including task IDs in commit messages or branch names.

## 2.5 Constraints

- SMS cost and delivery reliability vary by provider and region.
- Git provider API rate limits and permission models.
- Compliance with company security policy and applicable data-protection law.
- Budget, team size and delivery timeline.

# Functional Requirements

Priority: **H** = must have, **M** = should have, **L** = nice to have.

Phase: **P1** = MVP, **P2** = second release, **P3** = later release (Refer release plan for a better understanding of the phase).

| ID | Requirement | Priority | Phase |
|---|---|---|---|
| FR-1 | The system shall be architected as an internal company platform where all users, teams, and projects operate within a single company scope with role-based access control. | H | P1 |
| FR-2 | The system shall allow users to register by invitation and log in with email and password. | H | P1 |
| FR-3 | The system shall support multi-factor authentication. | M | P2 |
| FR-4 | The system shall support single sign-on through SAML 2.0 or OpenID Connect. | M | P2 |
| FR-5 | The system shall implement role-based access control with the roles in Section 2.2 and configurable permissions. | H | P1 |
| FR-6 | The system shall assign each developer a specialty (Frontend, Backend, UI/UX) that identifies them to teammates. | H | P1 |
| FR-7 | The system shall let users manage their profile, phone number, Telegram account, GitHub account, time zone and notification preferences. | H | P1 |
| FR-8 | The system shall enforce password policy, account lockout and session expiry, and support password reset. | H | P1 |
| FR-9 | The system shall allow admins to deactivate users and reassign their open work. | H | P1 |

## 3.2 Project Management

| ID | Requirement | Priority | Phase |
|---|---|---|---|
| FR-10 | The system shall allow a manager to create a project with title, detailed description, goals, scope, tech stack, milestones, start date and final deadline. | H | P1 |
| FR-11 | The system shall allow a project to be edited, put on hold, archived or closed, with a recorded reason. | H | P1 |
| FR-12 | The system shall support project templates for recurring project types. | L | P3 |
| FR-13 | The system shall support milestones with deadlines and completion tracking. | M | P2 |
| FR-14 | The system shall allow documents and links to be attached to a project. | M | P2 |
| FR-15 | The system shall provide a portfolio view of all projects with status, health and progress. | H | P1 |

## 3.3 Team Management

| ID | Requirement | Priority | Phase |
|---|---|---|---|
| FR-16 | The system shall allow creation of teams within a project (Frontend, Backend, UI/UX and custom teams). | H | P1 |

| ID | Requirement | Priority | Phase |
|---|---|---|---|
| FR-17 | The system shall allow members to be added to or removed from teams, and a Team Lead to be designated. | H | P1 |
| FR-18 | The system shall show each person's team, role and contact information to other members of the project. | H | P1 |
| FR-19 | The system shall allow a deadline to be set for each team. | H | P1 |
| FR-20 | The system shall allow a person to belong to multiple projects and teams and show their combined workload. | M | P2 |

## 3.4 Task Breakdown and Assignment

| ID | Requirement | Priority | Phase |
|---|---|---|---|
| FR-21 | The system shall allow tasks and sub-tasks to be created with title, description, acceptance criteria, priority, estimate, deadline and labels. | H | P1 |
| FR-22 | The system shall support AI-assisted task breakdown suggestions generated via private, internal infrastructure (e.g., private AWS Bedrock VPC endpoint) with mandatory manager review and approval before any task is created. | M | P2 |
| FR-23 | The system shall assign each task to a specific person within a team (for example, Person X in the Frontend Team does tasks A, B and C). | H | P1 |
| FR-24 | The system shall support task statuses To Do, In Progress, In Review, Done and Blocked, with configurable workflow rules. | H | P1 |
| FR-25 | The system shall support task dependencies (task B cannot start until task A is done): warn or block when someone starts a task whose dependency is not finished, reject circular dependencies, notify the assignee when a dependency is completed, and warn when a dependency threatens a deadline. | H | P1 |
| FR-26 | The system shall allow reassignment and notify the previous and new assignee. | H | P1 |
| FR-27 | The system shall balance workload: show each person’s assigned effort against capacity, warn when someone is over capacity, and suggest less-loaded teammates when assigning a task. | H | P1 |
| FR-28 | The system shall support comments, @mentions and attachments on tasks. | M | P2 |
| FR-29 | The system shall record the full history of changes to each task. | H | P1 |

## 3.5 Skills Profiles and Automatic Task Assignment

| ID | Requirement | Priority | Phase |
|---|---|---|---|
| FR-71 | The system shall require every developer to complete a skills profile on first login (skills, proficiency level, years of experience, preferred technologies) before they can be assigned work. | H | P1 |
| FR-72 | The system shall provide a managed skill catalog grouped by specialty (for example Angular, TypeScript, Node.js, Express, databases, Figma, UX research) from which developers choose; developers may propose new skills, which an Admin or manager approves. | H | P1 |
| FR-73 | The system shall record a proficiency level per skill (Beginner, Intermediate, Advanced, Expert) and prompt developers to review their skills periodically. | H | P1 |

| ID | Requirement | Priority | Phase |
|---|---|---|---|
| FR-74 | The system shall let each task carry required skills and minimum proficiency, entered by the manager or extracted from the task description by the AI breakdown feature. | H | P1 |
| FR-75 | The system shall automatically recommend or make the assignment by matching a task's required skills to developers' skills and proficiency, taking into account current workload, availability (leave, working hours), the task deadline and the developer's team and specialty. | H | P1 |
| FR-76 | The system shall support two assignment modes, configurable per project: automatic assignment, and suggested assignment that requires manager approval. The manager can always override or reassign. | H | P1 |
| FR-77 | The system shall show for each recommended assignee a match score and a plain-language explanation (for example, matches Angular and TypeScript, 2 open tasks). | M | P1 |
| FR-78 | The system shall distribute work fairly and shall not assign tasks to a developer who is over capacity (see FR-27). | H | P1 |
| FR-79 | The system shall flag a skill gap to the manager when no available developer meets a task's skill requirement, and propose alternatives (closest match, pairing, reassigning, training). | M | P1 |
| FR-80 | The system shall allow a Team Lead or manager to confirm or adjust a developer's declared skill levels. | M | P1 |
| FR-81 | The system shall use evidence from Git activity (for example languages and technologies in a developer's merged pull requests) to suggest skill updates. | L | P2 |
| FR-82 | The system shall record the reason for every automatic or suggested assignment in the task history and audit log, and notify the assignee. | H | P1 |
| FR-83 | The system shall provide a skills and skill-gap report across teams and projects. | M | P2 |

## 3.6 Git Integration

| ID | Requirement | Priority | Phase |
|---|---|---|---|
| FR-30 | The system shall integrate with GitHub using an approved GitHub App or OAuth, with least-privilege scopes. | H | P1 |
| FR-31 | The system shall allow a project to be linked to one or more repositories. | H | P1 |
| FR-32 | The system shall require each developer to link a GitHub identity so activity can be attributed to them. | H | P1 |
| FR-33 | The system shall receive repository events by webhook (commits, branches, pull requests, reviews, merges) with signature verification, and reconcile through the API if events are missed. | H | P1 |
| FR-34 | The system shall associate commits, branches and pull requests with tasks using a task ID (for example TASK-123) in the message or branch name. | H | P1 |
| FR-35 | The system shall display per developer and per task the commits, pull requests, review state and merge status. | H | P1 |
| FR-36 | The system shall flag tasks as at risk when the deadline is near and no related activity exists. | M | P2 |

| ID | Requirement | Priority | Phase |
|---|---|---|---|
| FR-37 | The Git integration shall be built behind a provider abstraction so GitLab and Bitbucket can be added. | M | P3 |

## 3.7 Review

| ID | Requirement | Priority | Phase |
|---|---|---|---|
| FR-38 | The system shall give controllers and reviewers a consolidated view of developer code changes across projects. | H | P1 |
| FR-39 | The system shall allow reviewers to approve, reject or request changes with comments, and mirror the decision to the pull request where permitted. | M | P2 |
| FR-40 | The system shall notify developers of review outcomes and alert reviewers of pending reviews, including those open longer than a configurable time. | H | P1 |
| FR-41 | The system shall prevent a task being marked Done until the required review state is met (configurable per project). | M | P2 |

## 3.8 Progress Updates

| ID | Requirement | Priority | Phase |
|---|---|---|---|
| FR-42 | The system shall let a manager configure the update interval per project or team (for example every 3 or 7 days). | H | P1 |
| FR-43 | The system shall prompt each developer for an update at the interval, with fields for work done, next steps, blockers and confidence level. | H | P1 |
| FR-44 | The system shall allow updates to be submitted from the web app or by replying to the Telegram bot. | M | P2 |
| FR-45 | The system shall flag missed updates and notify the Team Lead and manager. | H | P1 |
| FR-46 | The system shall let a developer raise a blocker (“I’m stuck”) with a short description, which sets the task to Blocked and immediately notifies the Team Lead and manager. | H | P1 |
| FR-47 | The system shall produce a team digest summarizing updates for each cycle. | M | P2 |

## 3.9 Notification Engine

| ID | Requirement | Priority | Phase |
|---|---|---|---|
| FR-48 | The system shall deliver notifications through Telegram and SMS, and also through email and in-app. | H | P1 |
| FR-49 | The system shall send notifications for assignments, changes, review events, blockers and mentions. | H | P1 |
| FR-50 | The system shall send deadline reminders at configurable offsets (for example 3 days, 1 day, same day) and overdue alerts. | H | P1 |
| FR-51 | The system shall support escalation rules (for example notify the Team Lead after 1 day overdue and the manager after 3 days). | M | P2 |

| ID | Requirement | Priority | Phase |
|---|---|---|---|
| FR-52 | The system shall let each user choose their preferred reminder channel (Telegram, SMS, email, in-app or any combination) per notification type, and set quiet hours in their time zone. | H | P1 |
| FR-53 | The system shall use editable message templates with variables and language support. | M | P2 |
| FR-54 | The system shall queue notifications, retry on failure, fall back to another channel if configured, and record delivery status. | H | P1 |
| FR-55 | The system shall respect consent and allow users to opt out of non-critical channels, and shall avoid duplicate messages. | H | P1 |
| FR-56 | The system shall automatically generate and send a weekly report to managers (and optionally team leads and executives) covering progress per project, team and person, completed and overdue tasks, blockers raised, missed updates, at-risk tasks and workload, delivered by email, Telegram and as a downloadable PDF. | H | P1 |

## 3.10 Dashboards and Reporting

| ID | Requirement | Priority | Phase |
|---|---|---|---|
| FR-57 | The system shall show managers progress by project, team and person. | H | P1 |
| FR-58 | The system shall list overdue, at-risk and blocked tasks. | H | P1 |
| FR-59 | The system shall give developers a personal dashboard of tasks, deadlines and pending updates. | H | P1 |
| FR-60 | The system shall provide delivery metrics such as on-time rate, cycle time, update compliance and workload distribution. | M | P2 |
| FR-61 | The system shall provide burndown and timeline (Gantt-style) views. | L | P3 |
| FR-62 | The system shall export reports to PDF, CSV and Excel, and support scheduled report delivery. | M | P2 |

## 3.11 Audit and Compliance

| ID | Requirement | Priority | Phase |
|---|---|---|---|
| FR-63 | The system shall keep an immutable audit log of security-relevant and administrative actions (who, what, when, from where). | H | P1 |
| FR-64 | The system shall let admins search and export audit logs. | M | P2 |
| FR-65 | The system shall support data retention settings and the deletion or anonymization of a user's personal data on request. | M | P2 |

## 3.12 Administration and Integrations

| ID | Requirement | Priority | Phase |
|---|---|---|---|
| FR-66 | The system shall provide an admin console for users, roles, teams, integrations, notification providers and policies. | H | P1 |

| ID | Requirement | Priority | Phase |
|---|---|---|---|
| FR-67 | The system shall provide a versioned REST API with token-based authentication and rate limiting. | M | P2 |
| FR-68 | The system shall send outgoing webhooks for key events so other tools can subscribe. | L | P3 |
| FR-69 | The system shall allow import of projects and tasks from CSV and, later, from other tools (for example Jira). | L | P3 |
| FR-70 | The system shall show system and integration health (queue backlog, failed notifications, webhook errors) to admins. | M | P2 |

# Use Cases

## 4.1 Use Case List

| ID | Use case | Actor |
|---|---|---|
| UC-1 | Sign in (including SSO) | All users |
| UC-2 | Create project and milestones | Manager |
| UC-3 | Create teams and add members | Manager |
| UC-4 | Break down project into tasks | Manager |
| UC-5 | Assign task to developer (skill-based, automatic or approved) | System, Manager, Team Lead |
| UC-15 | Complete skills profile | Developer |
| UC-6 | Link repository and GitHub identity | Admin, Manager, Developer |
| UC-7 | View developer code activity | Manager, Reviewer |
| UC-8 | Review pull request / task | Reviewer |
| UC-9 | Submit progress update | Developer |
| UC-10 | Receive reminder and escalation | Developer, Team Lead, Manager |
| UC-11 | Raise a blocker | Developer |
| UC-12 | View dashboards and reports | Manager, Executive |
| UC-13 | Configure notifications and policies | Admin, Manager |
| UC-14 | Audit review | Admin |

## 4.2 Detailed Use Cases

### UC-1: Assign Task to Developer

| Field | Description |
|---|---|
| Actor | System (automatic mode), Manager or Team Lead |
| Precondition | Project, team and task exist; the developer is a member of a team in the project. |

| Field | Description |
|---|---|
| Main flow | 1. Task is created with required skills. 2. System matches skills, workload, availability and deadline and recommends a developer with an explanation. 3. In approval mode the manager confirms or overrides; in automatic mode the system assigns directly. 4. System checks the developer's workload. 5. System saves the assignment, records the reason and writes an audit entry. 6. System notifies the developer by their preferred channels. |
| Alternative flows | A: Developer is over capacity, so the system warns and the actor can proceed or choose another person. B: Notification fails, so the system retries and falls back to another channel. |
| Postcondition | Task shows assignee and deadline; developer is notified; history is recorded. |
| Related requirements | FR-23, FR-26, FR-27, FR-29, FR-49, FR-54, FR-71 to FR-82 |

### UC-2: View Developer Code Activity

| Field | Description |
|---|---|
| Actor | Manager, Reviewer |
| Precondition | Repository is linked and the developer has linked a GitHub identity. |
| Main flow | 1. Actor opens a project and selects a developer or task. 2. System shows linked commits, pull requests, review and merge status. |
| Alternative flows | A: Webhook events were missed, so the system reconciles through the API. B: Provider is unavailable, so the system shows the last synchronized data with a warning. |
| Postcondition | Actor sees current, task-linked code activity. |
| Related requirements | FR-32 to FR-35, FR-38 |

### UC-3: Submit Progress Update

| Field | Description |
|---|---|
| Actor | Developer |
| Precondition | Developer has active tasks and an update is due. |
| Main flow | 1. System sends a reminder. 2. Developer opens the form or replies in Telegram. 3. Developer enters work done, next steps, blockers and confidence. 4. System saves it and updates the team digest. |
| Alternative flows | A: Developer reports a blocker, so the manager is notified immediately. B: Update is missed, so the system escalates per policy. |
| Postcondition | Update is recorded and visible on dashboards. |
| Related requirements | FR-42 to FR-47, FR-51 |

## Non-Functional Requirements

Targets below are proposed and must be confirmed with stakeholders and operations.

| ID | Category | Requirement |
|---|---|---|
| NFR-1 | Performance | 95% of API requests respond within 500 ms and dashboard pages load within 2 seconds under the target load. |

| ID | Category | Requirement |
|---|---|---|
| NFR-2 | Scalability | Architected to support up to 50 concurrent internal engineering users, 1,000 active projects, and 50,000 tasks on AWS infrastructure without performance degradation. |
| NFR-3 | Availability | Monthly availability of at least 99.9% excluding planned maintenance. |
| NFR-4 | Disaster recovery | Recovery point objective of 15 minutes or less and recovery time objective of 4 hours or less; automated daily backups with tested restores. |
| NFR-5 | Security | Follow OWASP ASVS Level 2; TLS 1.2 or higher in transit; AES-256 encryption at rest; secrets and tokens held in a secrets manager; encrypted storage of Git tokens. |
| NFR-6 | Tenant isolation | Enforce strict referential integrity and role-based permission across all the projects, teams and tasks. |
| NFR-7 | Access control | Least privilege by default; all access checks enforced on the server. |
| NFR-8 | Privacy | Personal data (phone, Telegram ID, email) minimized, access-controlled, and handled according to applicable data-protection law and company policy. |
| NFR-9 | Reliability of notifications | Notification delivery is at-least-once with de-duplication; failed deliveries retried with back-off; 99% of reminders sent within 1 minute of schedule. |
| NFR-10 | Observability | Centralized structured logs, metrics, tracing and alerting for API, workers, queues and integrations. |
| NFR-11 | Maintainability | Modular architecture, automated tests (target at least 80% coverage on core logic), code review, and CI/CD. |
| NFR-12 | Usability | Responsive UI; WCAG 2.1 AA accessibility; a new user completes core tasks without training. |
| NFR-13 | Localization | Support for time zones, date formats and multiple languages (language list to be confirmed). |
| NFR-14 | Compatibility | Supported on current and previous major versions of mainstream browsers. |
| NFR-15 | Auditability | Audit records are tamper-evident and retained for a configurable period. |

# External Interface Requirements

## 6.1 User Interface

Main screens: Sign in, Portfolio dashboard, Project detail, Team view, Task board and list, Developer dashboard, Code activity and review, Progress update form, Notification preferences, Reports, Admin console, Audit log. Wireframes will be produced separately [reference to be added].

## 6.2 Software Interfaces

| Service | Purpose | Notes |
|---|---|---|
| GitHub (App / OAuth, REST and webhooks) | Repository activity, pull requests, reviews | Signature verification, rate limits, reconciliation job |

| Service | Purpose | Notes |
|---|---|---|
| Telegram Bot API | Reminders, alerts and updates by chat | User must start the bot to link their account |
| SMS gateway | SMS reminders and critical alerts | Provider to be selected; delivery receipts required |
| Email service | Email notifications and digests | SPF/DKIM configured |
| Identity provider (optional) | SSO via SAML 2.0 / OIDC | Phase 2 |
| AI / LLM service (optional) | Task breakdown suggestions | Human approval required; no sensitive data sent without policy approval |

### 6.3 Communication and API Standards

- HTTPS only; REST with JSON; versioned endpoints (for example /api/v1).
- Token-based authentication; rate limiting; consistent error format.
- Outgoing webhooks signed with a shared secret.

## Data Requirements

| Entity | Key attributes |
|---|---|
| Organization | id, name, settings, plan, created at |
| User | id, org id, name, email, role, specialty, phone, telegram ID, GitHub identity, time zone, status |
| Project | id, org id, title, description, goals, tech stack, start date, deadline, update interval, status, health |
| Milestone | id, project id, title, deadline, status |
| Team / TeamMember | id, project id, name, type, lead id, deadline; member user id, role |
| Skill / UserSkill | skill: id, name, specialty, status; user skill: user id, skill id, proficiency, years, confirmed by |
| TaskRequiredSkill | task id, skill id, minimum proficiency |
| Task | id, project id, team id, assignee id, parent id, title, description, acceptance criteria, priority, estimate, status, deadline |
| TaskDependency | task id, depends on task id |
| Repository / GitActivity | repo id, project id, provider; activity: user id, task id, type (commit, PR, review), reference, status, timestamp |
| Review | id, PR or task id, reviewer id, decision, comment, date |
| ProgressUpdate | id, user id, project id, cycle, work done, next steps, blockers, confidence, date |
| Notification / Delivery | id, user id, type, channel, template, status, attempts, sent at, delivered at |
| EscalationRule / Policy | id, scope, trigger, offset, target role, channel |
| AuditLog | id, org id, actor, action, entity, before/after, IP, timestamp |

### 7.1 Retention, Backup and Privacy

- Retention periods for activity data, notifications and audit logs are configurable per organization [defaults to be agreed].
- Daily full and continuous incremental backups; restores tested periodically.
- Personal data can be exported, anonymized or deleted on request.

## Release Plan

| Phase | Focus | Outcome |
|---|---|---|
| P1 (MVP) | Organizations, roles, projects, teams, developer skills profiles, skill-based task assignment, tasks with workflow (To Do, In Progress, In Review, Done), dependencies, workload balancing, assignment, blockers, GitHub linking and activity, progress updates, user-chosen Telegram/SMS/email reminders, weekly report, basic dashboards, audit log | A pilot team can plan, assign, track and be reminded end to end |
| P2 | SSO and MFA, AI task suggestions, escalations, review workflow, workload and risk analytics, reports and exports, public API | Broader rollout across teams |
| P3 | GitLab/Bitbucket, webhooks out, imports, Gantt/burndown, client portal, templates | Enterprise integration and expansion |

## Risks and Mitigation

| Risk | Impact | Mitigation |
|---|---|---|
| Developers do not link GitHub or follow the task ID convention | Poor traceability | Onboarding checks, commit-message hooks, dashboards showing unlinked work |
| SMS or Telegram delivery failures | Missed reminders | Retries, channel fallback, delivery status monitoring |
| Notification fatigue | Users ignore alerts | Preferences, quiet hours, digests, de-duplication |
| GitHub rate limits or API changes | Stale data | Webhooks first, caching, reconciliation, provider abstraction |
| Scope growth | Delay | Phased releases and change control |
| AI suggestions are wrong | Bad plans | Human approval required and editable output |
| Low adoption | No business value | Pilot team, feedback loop, training, executive support |

## Acceptance and Verification

- Each requirement shall be verified by test, inspection, demonstration or analysis, and tracked in a requirements traceability matrix [to be created].
- Each functional requirement shall have at least one acceptance criterion written in Given / When / Then form.

- The MVP is accepted when all P1 high-priority requirements pass user acceptance testing with the pilot team, with no open critical or high-severity defects.
- Non-functional targets are verified through load, security and recovery testing before production release.

## Open Questions for Stakeholders

| # | Question | Owner |
|---|---|---|
| 1 | Who are the intended users and how many teams, developers and projects at launch? Internal only or also for clients? | Internal only |
| 2 | Single company use or multi-tenant product to be sold or offered to other companies? | Single company use |
| 3 | Hosting: cloud provider or on-premises? Any data-residency requirements? | AWS , none |
| 4 | Preferred SMS provider and budget; is Telegram acceptable for company communication? | Ethiotelecom, Yes |
| 5 | Git provider(s) in use: only GitHub, or also GitLab/Bitbucket? GitHub organization or personal repositories? | Only Github |
| 6 | Existing tools that must be integrated or replaced (Jira, Slack, email, HR system)? | None |
| 7 | Compliance, security and privacy policies that apply? | HTTPS, hashed passwords, encrypted tokens, role-based access, audit log. |
| 8 | Required languages and time zones? | English first. Each user has their own time zone |
| 9 | Budget, team size, timeline and the definition of a successful MVP? | -, 3 to 4 dev, -, when it works perfectly for a developer |
| 11 | Should skill-based assignment be fully automatic or always approved by a manager? Who verifies developers' declared skills? | Be automatic but the manager have to see it |
| 10 | Is AI task breakdown desired, and is sending project descriptions to an external AI service allowed? | Yes, no |

## Appendix A: Glossary

| Term | Meaning |
|---|---|
| SRS | Software Requirements Specification. |
| Controller | The manager role that oversees projects and monitors developers. |
| Tenant / Organization | An isolated customer or company space within the platform. |
| Commit | A saved set of code changes in a Git repository. |
| Pull request (PR) | A request to merge code changes into another branch, usually after review. |

| Term | Meaning |
|---|---|
| Webhook | An automatic message sent from one system to another when an event happens. |
| RBAC | Role-based access control. |
| SSO / SAML / OIDC | Single sign-on and the standards used to implement it. |
| MFA | Multi-factor authentication. |
| RPO / RTO | Recovery point objective (acceptable data loss) and recovery time objective (acceptable downtime). |
| Blocker | Something preventing a developer from progressing on a task. |
| MVP | Minimum viable product: the first release that delivers core value. |

# Conclusion

This system closes the gap between planning and actual delivery. Today, managers don't know who is working on what, deadlines slip without warning, and nobody can see real code progress in one place. A manager creates a project, and the system breaks it into tasks. Each task goes to the right developer based on the skills they declared, their workload and their availability. GitHub activity shows whether the work is really moving. Reminders reach each person on the channel they chose, and blockers, overdue tasks and missed updates go straight to the people who need to act on them. A weekly report keeps management informed without anyone chasing updates.

The requirements in this document are phased so the first release already delivers the core value: clear ownership, skill-based assignment, visible progress and timely reminders. Security, auditability and scalability are built in from the start, so the system can grow from one team to the whole company.